UCS761 - Deep Learning Lab 5

Lab 5: Linear Regression Neuron: Learning by Gradient Descent (No ML Libraries)

Dataset

Abalone dataset (UCI):

https://archive.ics.uci.edu/dataset/1/abalone

Target

$$y = Rings + 1.5$$

Inputs: Use only numeric measurements (you will decide which ones, but must justify).

What you are building

A single linear neuron that predicts a numeric value.

With three input features, the model looks like:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

More generally, with `d` features:

$$\hat{y} = w_1x_1 + w_2x_2 + ... + w_dx_d + b$$

Where:

- `d` = number of input features you chose
- `w_1, w_2, ..., w_d` = learnable coefficients
- `b` = learnable bias

Part A: Data Setup (you do this, no shortcuts)

A1. Load dataset

You must load the data into a table and print:

- number of rows
- column names
- first 5 rows

Checkpoint comment in code:

- what is input
- what is output
- why output is numeric

```
# TODO: load dataset
# TODO: print number of rows
# TODO: print column names
# TODO: print first 5 rows
# Checkpoint:
# what is input:
# what is output:
# why output is numeric:
```

A2. Convert target

Create:

$$y = Rings + 1.5$$

```
# TODO: create target y = Rings + 1.5
```

A3. Choose features (important)

Pick exactly **3 numeric features** to start (same spirit as DLQ4).

Write a short justification comment:

- why those 3

- what you think each contributes

```
# TODO: select exactly 3 numeric features
# Justification:
# Feature 1:
# Feature 2:
# Feature 3:
```

A4. Train-test split

Do a simple 80/20 split (no sklearn).

Print the shapes.

```
# TODO: split data into train and test (80/20)
# TODO: print shapes
```

A5. Normalize inputs (mandatory)

Normalize using **training mean and std only**.

Checkpoint comment:

- why normalization is needed for learning

```
# TODO: compute mean and std from training data# TODO: normal
ize train and test using training stats# Checkpoint:# why nor
malization is needed for learning:
```

Part B: Define the model (write your own code)

Write a function that computes:

$$\hat{y} = Xw + b$$

Meaning, for one example:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Where:

- `X` has shape `(N, d)`
- `w` has shape `(d, 1)`
- `b` is a scalar

Required: You must print the shapes once.

Checkpoint comment:

- what are parameters here
- how many parameters exist for your chosen `d`

```
defforward(X, w, b):
    """
    computes y_hat = Xw + b
    """# TODOreturn y_hat
    # TODO: print shapes of X, w, b, y_hat
    # Checkpoint:
    # parameters are:
    # number of parameters:
```

Part C: Define loss (MSE)

Use Mean Squared Error.

Written out explicitly:

```
L = (1 / N) * [ (y_1 - y_hat_1)^2
              + (y_2 - y_hat_2)^2
              + ...
              + (y_N - y_hat_N)^2 ]
```

Write:

- `mse(y, y_hat)` function

Checkpoint comment:

- why square
- what kind of mistakes become expensive under MSE

```
defmse(y, y_hat):
# TODOreturn loss
# Checkpoint:
# why square:
# what mistakes are expensive:
```

Part D — The learning rule (you will derive + code)

This is the first time you will implement learning.

D1. Define “error”

For each example:

$$e = y_hat - y$$

D2. You need gradients

Your goal is to update:

- `w`
- `b`

Using the idea:

$$w = w - lr * dL_dwb = b - lr * dL_db$$

You are not required to write calculus derivation, but you must write in comments:

- what “gradient” means in words (slope / direction of increase)
- why subtracting the gradient reduces loss

```
# Checkpoint:
# what gradient means in words:
```

```
# why subtracting gradient reduces loss:
```

D3. Implement gradients (fill in)

Write two functions:

```
defgrad_w(X, y, y_hat):
    """
    returns gradient with respect to w
    shape must match w
    """# TODOreturn dWdefgrad_b(y, y_hat):
    """
    returns gradient with respect to b
    """# TODOreturn db
```

Constraint:

- confirm returned shapes match `w` and `b`

Checkpoint comment:

- if gradient is large, what does that mean
- what happens if learning rate is too large

```
# Checkpoint:
# meaning of large gradient:
# effect of too-large learning rate:
```

Part E: Training loop (minimal)

You will implement:

- forward pass
- loss
- gradients
- update

Skeleton to fill:

```
# initialize w (small random values)
# initialize b (zero)# choose learning rate and epochsfor epo
chinrange(epochs):
# 1) forward pass
# 2) compute loss
# 3) compute gradients
# 4) update w and bif epoch % k ==0:print(epoch, loss)
```

Checkpoint comment (keep the evolution):

- initial expectation: should loss go down fast or slow
- after seeing results: revise your expectation

```
# Initial expectation:# Revised expectation after training:
```

Part F: Evaluation (simple)

No confusion matrix.

No classification metrics.

Report:

- Test MSE
- Test MAE

Written explicitly:

```
MAE = (1 / N) * [ |y_1 - y_hat_1|
                + |y_2 - y_hat_2|
                + ...
                + |y_N - y_hat_N| ]
```

Print 5 examples:

- true age

- predicted age
- absolute error

```
# TODO: compute test MSE
# TODO: compute test MAE
# TODO: print 5 example predictions
```

Checkpoint comment:

- which cases seem systematically wrong (young or old)
- what DLQ4 prediction about "bias" you see in outputs

```
# Checkpoint:
# systematic errors:
# observed bias:
```

Deliverables

Submit one notebook or script containing:

- dataset loading and feature choice justification
- linear model forward pass
- MSE and MAE
- gradients and update rule
- training loop showing decreasing loss
- printed evaluation and 5 sample predictions
- reflective comments (do not rewrite earlier thoughts)

What you should not do

- do not use sklearn linear regression
- do not use torch or keras
- do not jump to hidden layers

- do not copy a formula without verifying shapes

“Today, you learned how a model updates numbers to reduce error.

Next lab, we will see how the same idea updates layers, not just one equation.”